

Nesne Yönelimli Analiz ve Tasarım

Analiz Aşaması

Gereksinimlerin Belirlenmesi

UML (Unified Modelling Language) Birleştirilmiş Modelleme Dili

Kullanım Durum (Use Case) Diyagramı

Sıralama Şeması

Nesne Yönelimli Analiz

- Analiz aşamasının ilk adımı gereksinimlerin belirlenmesidir.
- Gereksinimler, sistemin düzgün çalışması için yapması beklenenleri ifade eder
- Gereksinimler ilk olarak kullanıcılarla(paydaşlarla) görüşülerek (gerekli ise standart, yasa, yönetmelik vb. dokümanlar takip edilerek) oluşturulur, ve neler istendiği belirlenir.
- Daha sonra kullanım durumları (use case) belirlenerek gereksinimler ayrıntılandırılır.
- Sistemin tüm istenenlerle birlikte, beklenmeyen ya da istenmeyen durumları da göz önüne alınır.
- Kullanım durumları analiz edilerek nesneler, nesne üyeleri ve nesneler arasındaki bağıntılar belirlenir.

Modelleme

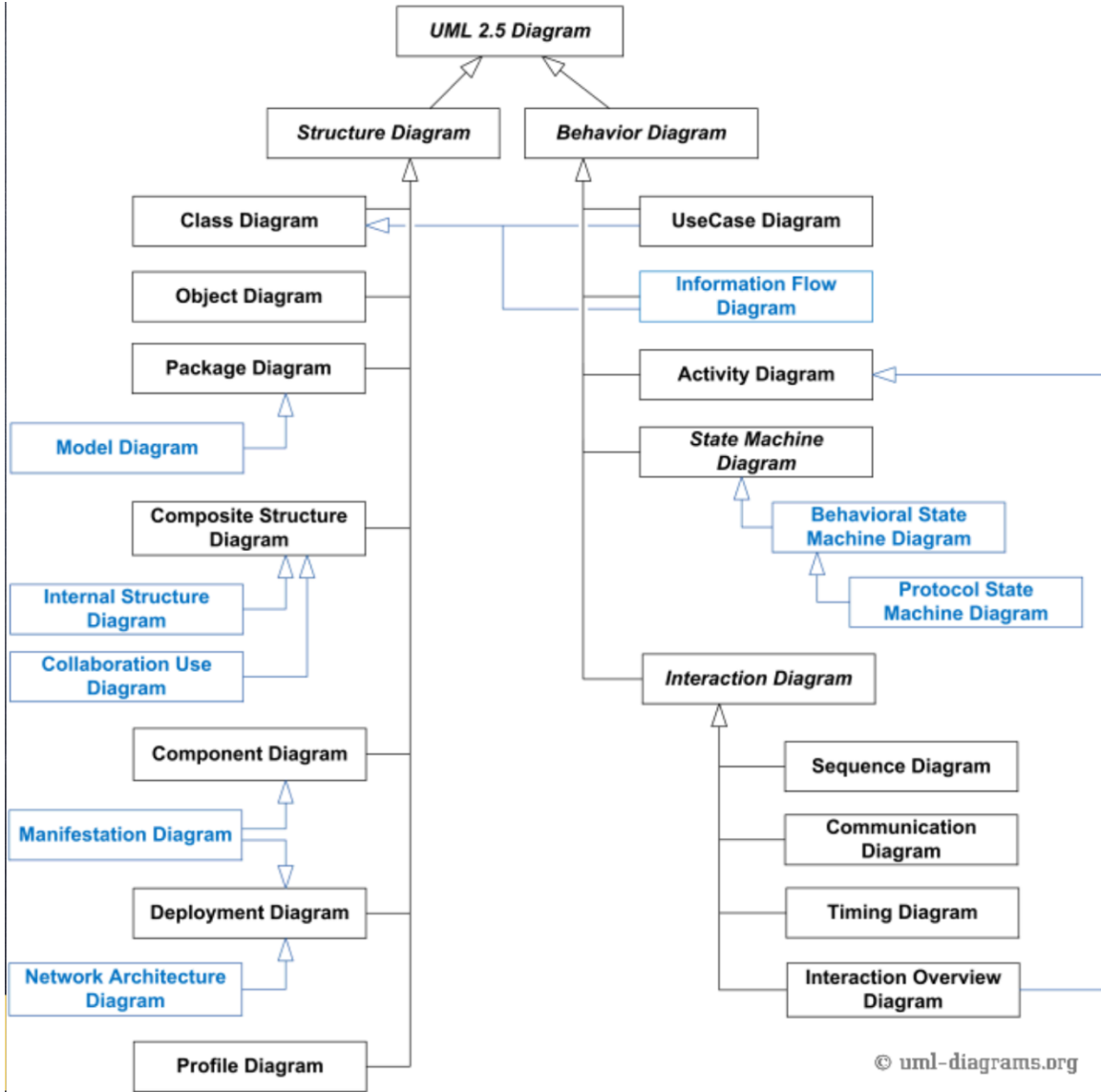
- Gerçeğin soyutunu (basit gösterimini) oluşturma işlemidir.
- Soyutlama sayesinde gereksiz ayrıntılar gizlenir ve ilgilenilecek detaylara odaklanılır. Böylece karmaşıklıktan kurtulunur.
- Ayrıntıların gerekli ya da gereksiz olduğu modelin amacına bağlıdır.
- Günümüz yazılımlarının kod karmaşıklığı her geçen gün artmaktadır.
- Bu nedenle, yazılımların farklı yönlerini, farklı ayrıntılarla ifade etmek üzere model oluşturulur.

Modelleme

- Model sayesinde; fikirlerin tartışılması/paylaşılması, yazılım aktörlerinin (son kullanıcı, yazılım geliştirici, tasarımcı, iş analisti vb.) iletişimi kolaylaşır. Yazılımın gerçekleştirilerek (kodlanması) ürün haline dönüştürülmesi daha kolay olur.

UML (The Unified Modeling Language)

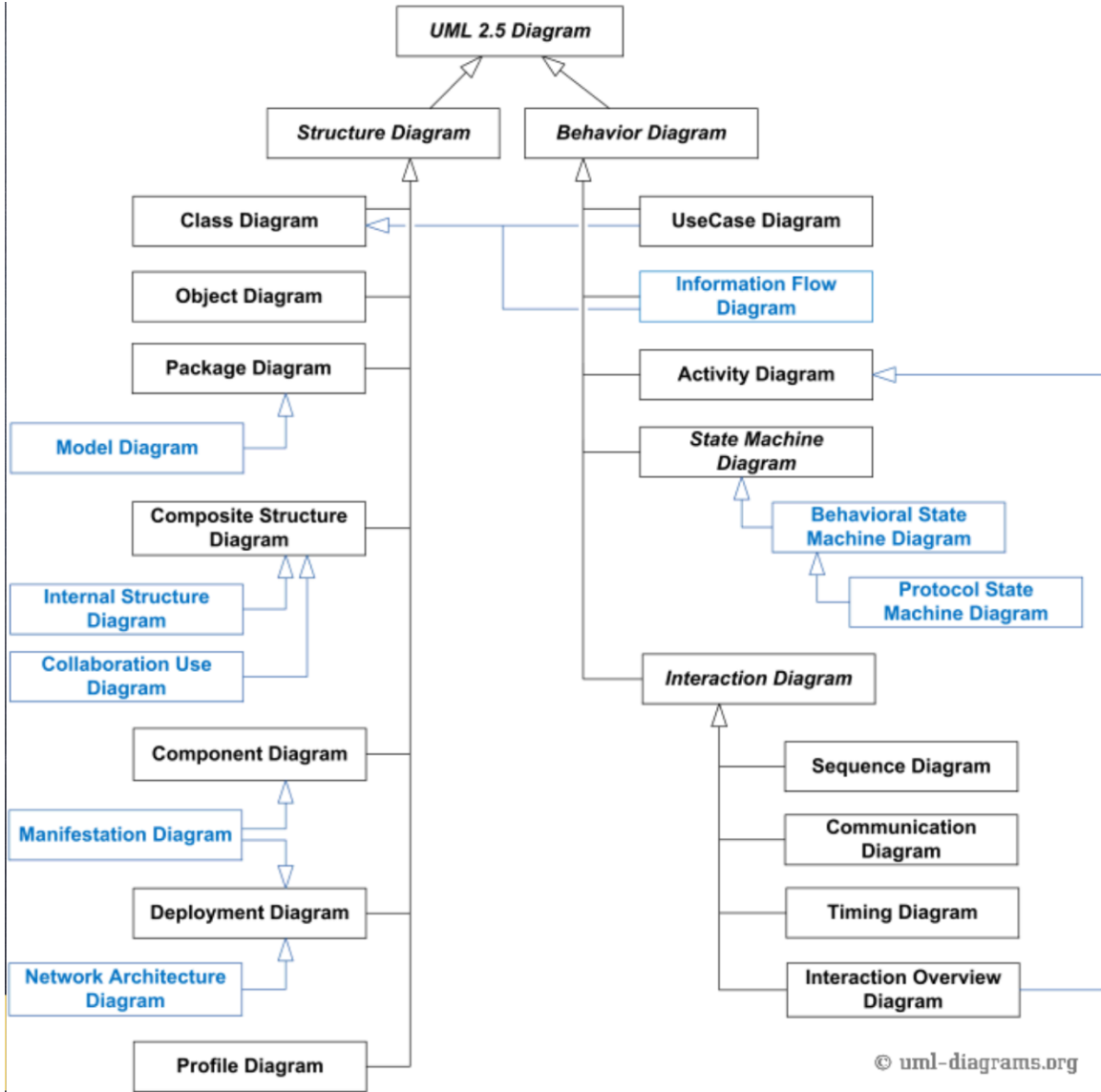
- Yazılım sistemlerinin temel bileşenlerini ifade etmek ve farklı yönlerini, farklı ayrıntılarla göstermek için kullanılan grafiksel modelleme dilidir.



- Yapı diyagramları:
Sistemin statik yapısını gösterir.
Sistem nedir?
Hangi parçalardan oluşur?

E-ticaret sistemi;

- Class diyagramı
- Nesne diyagramı
- Paket diyagramı



- Davranışsal diyagramlar:

Sistemdeki nesnelerin davranışlarını (zamana göre değişimlerini) gösterir.

Sistem ne yapar?

~~Parçalar nasıl etkileşir?~~

‘Para çekme’ ve ‘Sipariş Onay’ süreci

- Use case diyagramı
- Activity diyagramı
- Sequence diyagramı

Yapısal ve Davranışsal

Özellik	Yapısal Diyagramlar	Davranışsal Diyagramlar
Odak Noktası	Sistemin Organizasyonu	Sistemin Hareketleri
Zaman Faktörü	Zaman kavramı yoktur (Statik)	Zaman ve sıralama kritiktir (Dinamik)
Benzerlik	Bir binanın kat planı/projesi	Binadaki insanların günlük rotası
En Önemli Diyagram	Class Diagram	Use Case / Sequence Diagram

- Bir Kütüphane Otomasyonu düşünün. Kitapların listelenmesi **yapısal** mıdır yoksa **davranışsal** mı? Bir öğrencinin kitabı ödünç alıp süresini uzatması hangi diyagramla anlatılmalıdır?
- **Yapısal (Structural) Bakış:** Eğer odağımız "Bir kitabın hangi bilgileri (ad, yazar, ISBN, durum) olduğu" ve bu kitapların kütüphane sistemi içinde nasıl bir hiyerarşide durduğuna bakmaksa bu **Yapısaldır**.
- *Diyagram:* **Class (Sınıf) Diyagramı.** Burada "Kitap" bir sınıftır ve özellikleri statik olarak tanımlanır.
- **Davranışsal (Behavioral) Bakış:** Eğer odağımız "Kullanıcının butona basması, sistemin veritabanından kitapları çekmesi ve ekrana dökmesi" ise bu bir işlemdir ve **Davranışsaldır**.
- *Diyagram:* **Activity (Faaliyet) Diyagramı.** Listenin ekrana gelene kadar geçtiği adımları gösterir.

UML diyagramlarının süreçteki yeri

Süreç Aşaması	Diyagram Türü	Odak Noktası	Görseldeki Karşılığı
Gereksinim Analizi	Use Case Diagram	Sistemin sınırlarını ve aktörlerin (kullanıcıların) hedeflerini belirler.	<i>Behavioral -> UseCase</i>
Nesne Analizi	Class Diagram (Kavramsal)	Problemin içindeki temel kavramları (Öğrenci, Kitap, Ders) ve aralarındaki statik ilişkileri tanımlar.	<i>Structural -> Class</i>
Süreç Modelleme	Activity Diagram	Karmaşık iş akışlarını, karar noktalarını ve paralel süreçleri görselleştirir.	<i>Behavioral -> Activity</i>
Etkileşim Tasarımı	Sequence Diagram	Nesnelerin çalışma zamanında (runtime) birbirlerine nasıl mesaj gönderdiğini zaman ekseninde gösterir.	<i>Behavioral -> Interaction -> Sequence</i>
Detaylı Tasarım	State Machine Diagram	Tek bir nesnenin (Örn: Bir Sipariş nesnesi) yaşam döngüsü boyunca girdiği durumları (Hazırlanıyor, İptal, Teslim Edildi) inceler.	<i>Behavioral -> State Machine</i>
Mimari Tasarım	Component / Deployment	Sistemin fiziksel parçalarını ve hangi sunucularda çalışacağını belirler.	<i>Structural -> Component / Deployment</i>

Örnek Uygulama: ATM Mekanizması

(Yazılım Geliştirme Gereksinimlerin Belirlenmesi)

ATM aşağıdaki donanım bileşenlerinden oluşur:

- Tuş takımı; veri girişinden sorumludur.
- Ekran; mesajların görüntülenmesinden sorumludur.
- Para bölmesi; para alma ve para verme işleminden sorumludur.
- Kart bölmesi; banka kartını alınıp doğrulanmasından sorumludur.

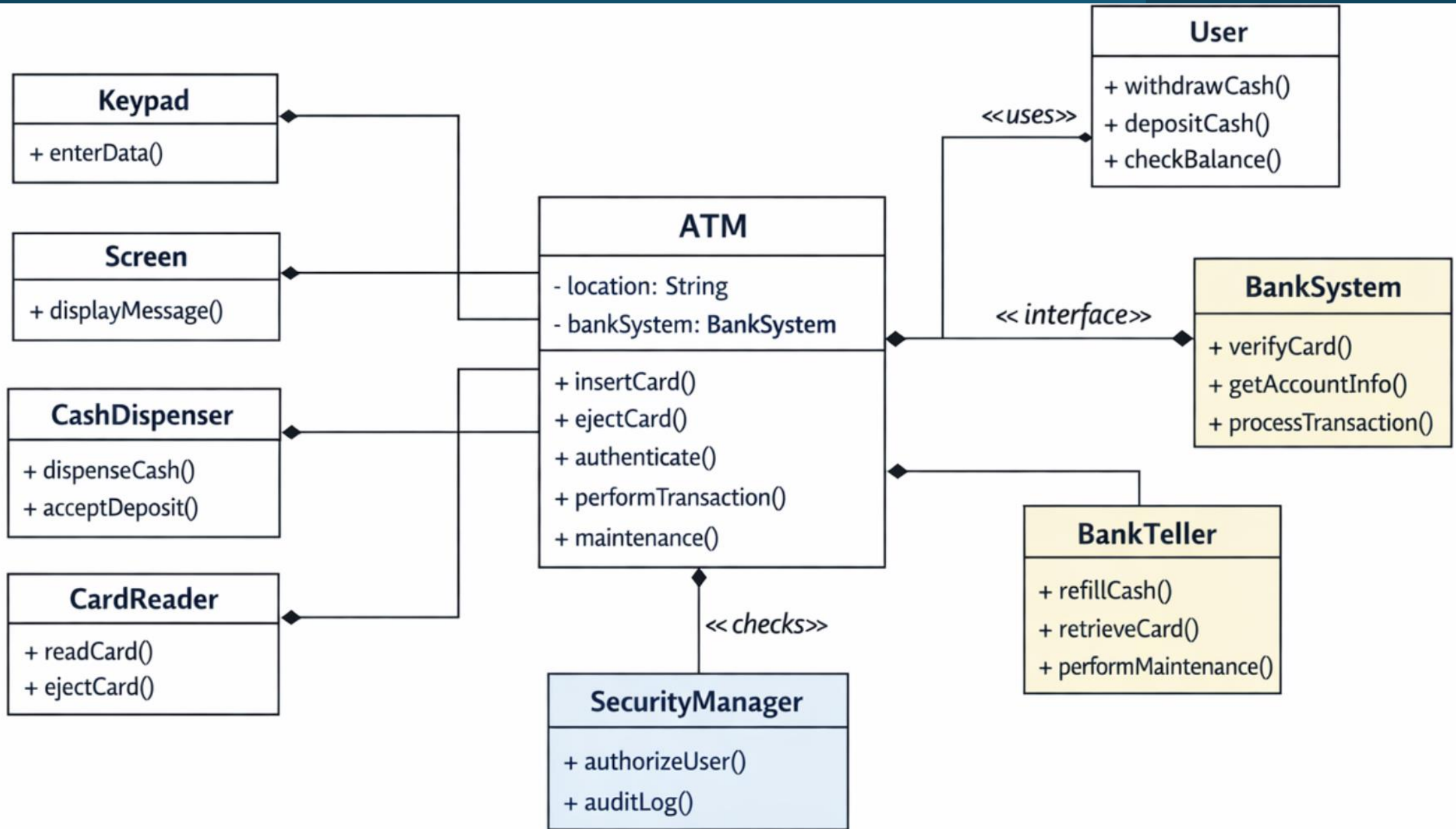
Kullanıcılar banka kartı ile para çekme, para yatırma, bakiye görüntüleme işlemlerini yapabilmelidirler.

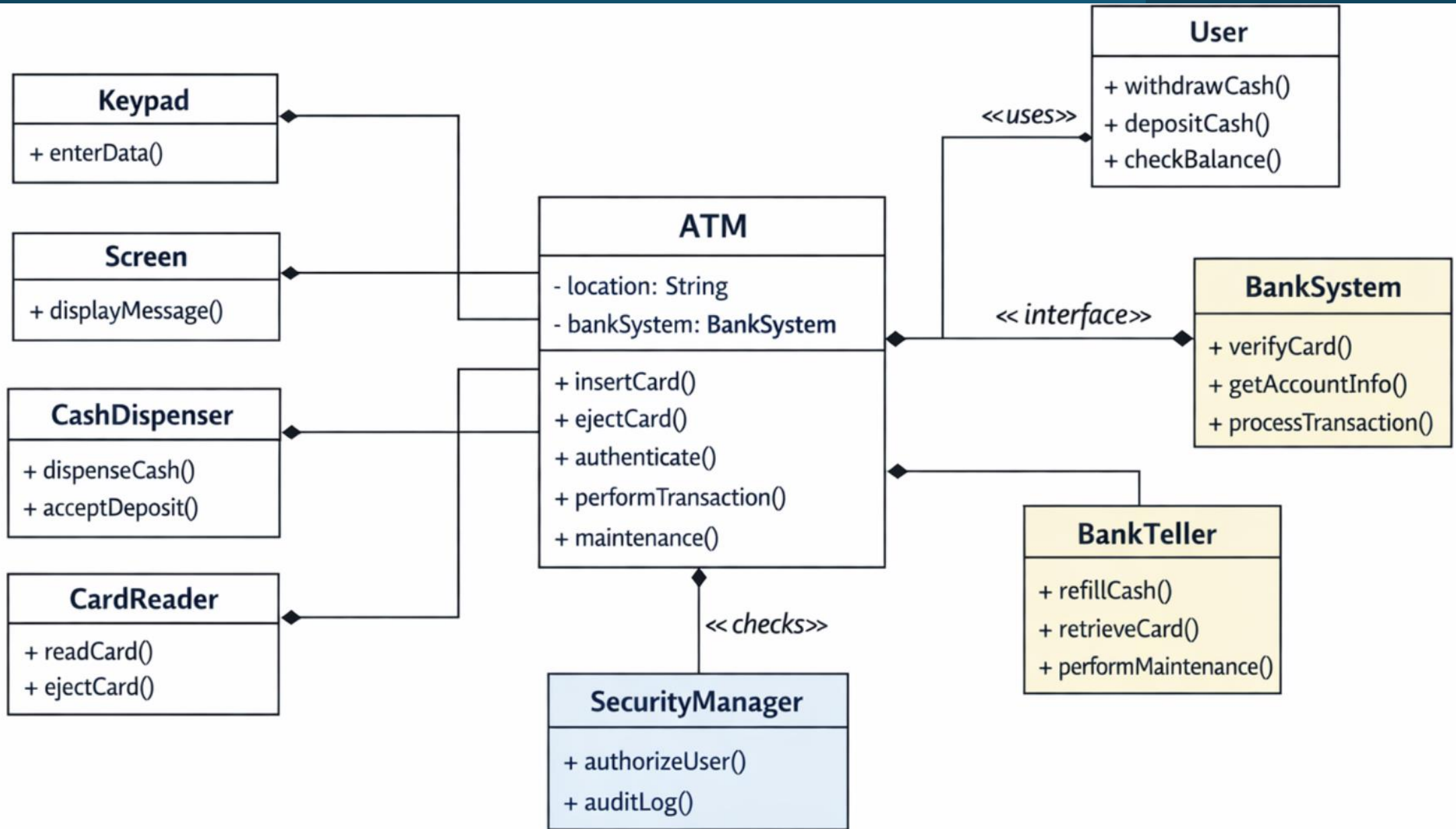
Banka görevlileri ATM sisteminin bakımını (para doldurma, sıkışan/yutulan kartları alma v.b.) yapmalıdır.

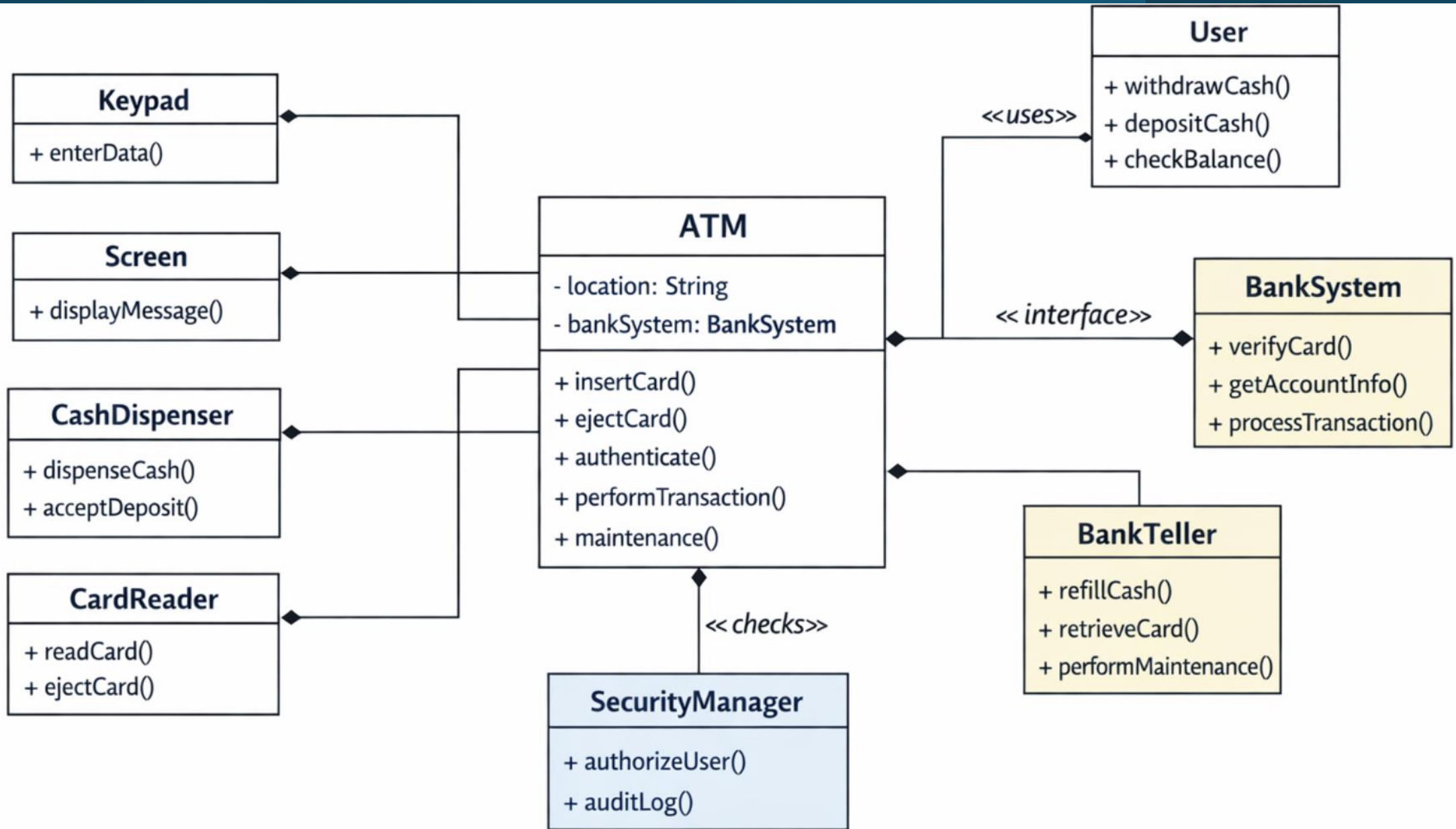
- Tüm işlemlerde güvenlik gereksinimleri göz önünde bulundurulmalıdır (yetki kontrolü).
- Sistem, Banka Bilgi Sistemiyle (kredi kartı servis sağlayıcıları ile) çevrimiçi çalışmalıdır.

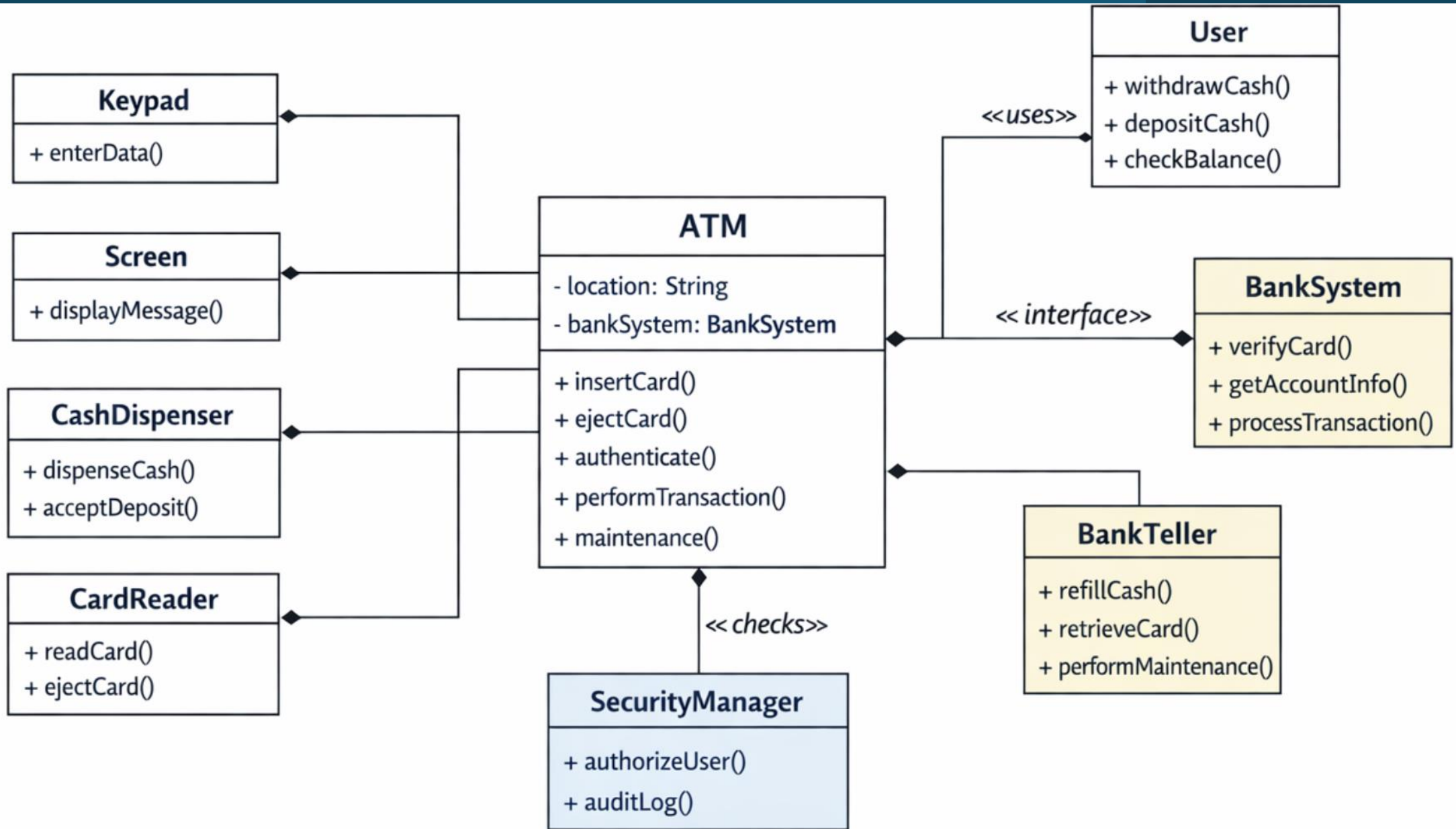
Yapısal Analiz

- Class Diyagramı: Donanım bileşenleri sınıflar olarak tanımlanır.
- İlişkiler: ATM sınıfı, donanım bileşenlerine sahiptir. ATM yoksa bu parçaların sistem içinde bir anlamı yoktur.
- Dış sistemler: BankaBilgiSistemi sınıfı ile ATM arasında bir bağlantı (Association) vardır.









Tasarım Prensipleri Açısından;

- Tek sorumluluk ilkesi
- Ayrık sorumluluklar

Composition and Aggregation

Özellik

Bağ Gücü

Parça tek başına var olabilir mi?

UML Sembolü

Örnek

Composition

Güçlü

 Hayır

Dolu elmas (◆)

ATM – Ekran

Aggregation

Zayıf

 Evet

Boş elmas (◇)

Üniversite – Öğrenci

- Araba-Motor
- Üniversite-Öğrenci

Davranışsal Analiz

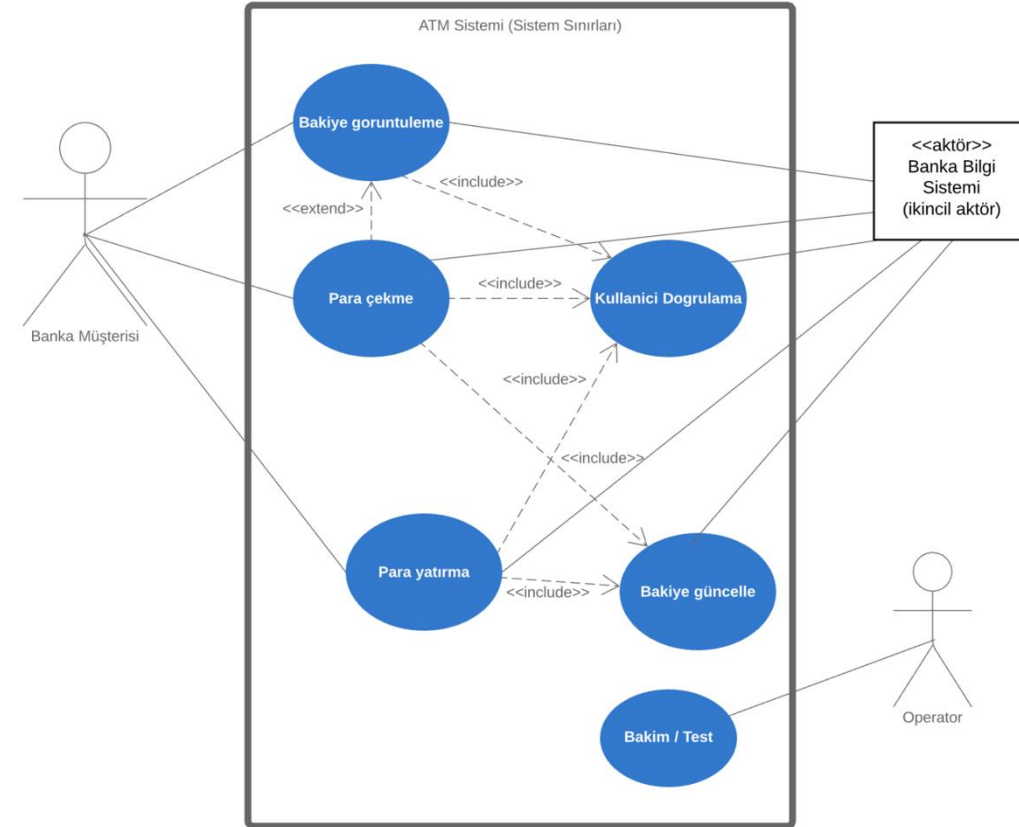
(Sistemin fonksiyonelliğini ve kullanıcı etkileşimini açıklar)

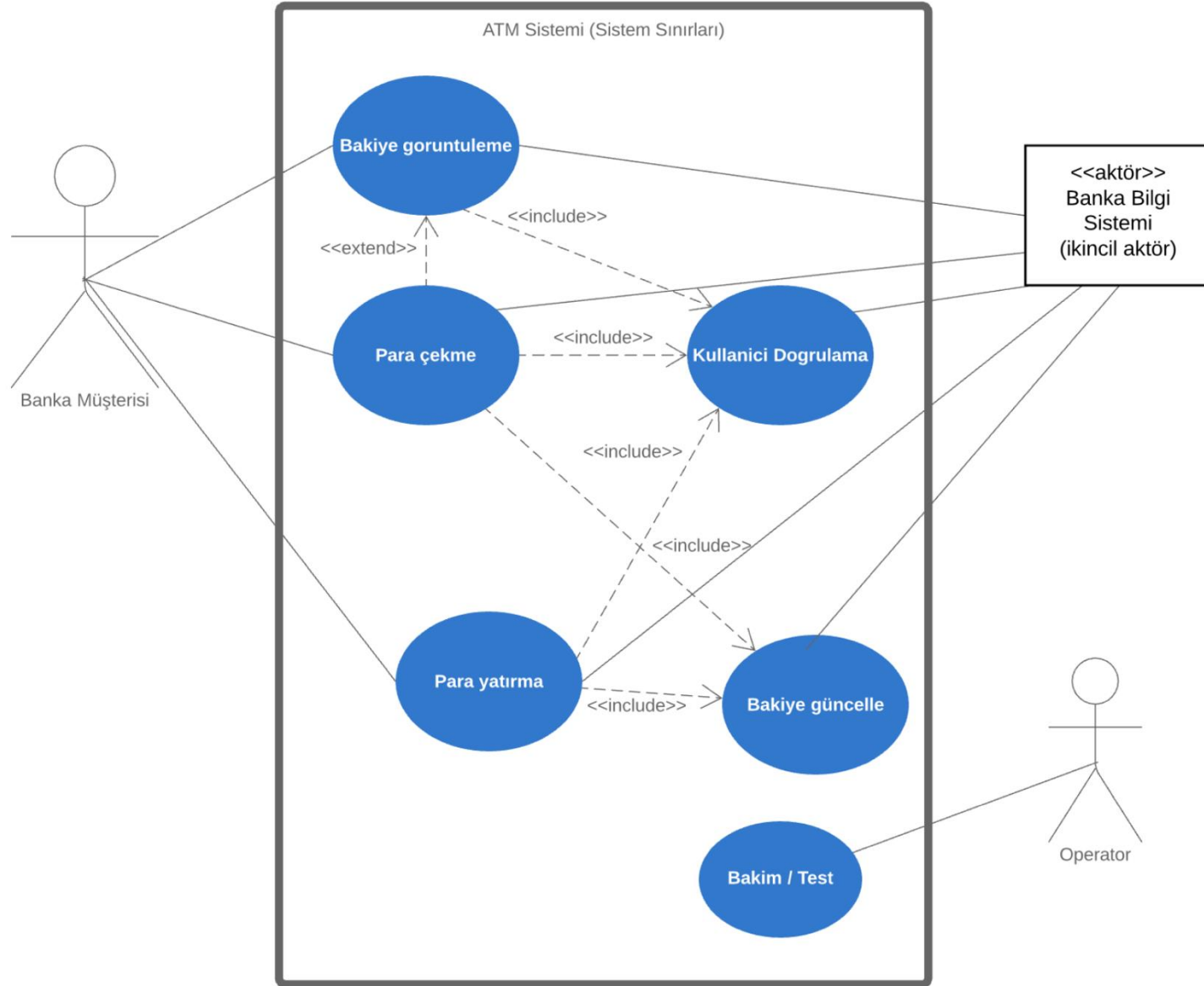
A. Use case diyagramı:

B. Sequence diyagramı:

Use case diyagramları

- Kullanım durumu, sistem ile kullanıcıların (aktör/rol) etkileşimini tanımlar.
- Her kullanım durumunun tek bir hedefi olmalıdır. Bu hedefe giden farklı yollar olabilir.
- Kullanım durumları bir araya gelerek (kullanım durumu senaryosu/modeli), aktör sistem ile etkileşime girdiğinde neler olduğunu anlatır.
- Kullanım durumları “sistem aktör açısından ne yapar” ile ilgilenir. Nasıl yapar ile ilgilenmez.
- Aktör belirli bir görev için olayları başlatan nesnedir. (Banka müşterisi, operator, banka v.s.)





Aktörler

1. Kullanıcı (Birincil Aktör)

- Sistemi başlatan aktördür.

Gerçek dünyada:

- Banka müşterisi

Başlatabildiği işlemler:

- Oturum Başlat
- Para Çekme
- Para Yatırma
- Çıkış

2. Operatör (Banka Görevlisi)

- Bakım işlemlerini başlatır.

Başlatabildiği işlemler:

- Para Yükleme
- Kart Geri Alma
- Sistem Bakımı

• 3. Banka Bilgi Sistemi (İkincil Aktör)

- Harici sistemdir.
- İşlemlerin tamamlanması için gereklidir.
- Kendisi işlem başlatmaz.
- Sistem tarafından çağrılır.
- Bu nedenle **ikincil aktör** olarak modellenmiştir.

Use case ayrıntıları

- Her kullanım durumunun tek bir hedefi olmalıdır. Bu hedefe giden farklı yollar olabilir.
- Kullanım durumunun ayrıntıları için, gereksinim listesi kullanılarak, metinsel ifadeler oluşturulur. Aktör ile sistem arasındaki olay akışı, istisnalar/ beklenmeyen durumlar da göz önüne alınarak anlatılır.

Kullanım durumu ayrıntılarında aşağıdakiler bulunur:

Kullanım durumunun tanımı:

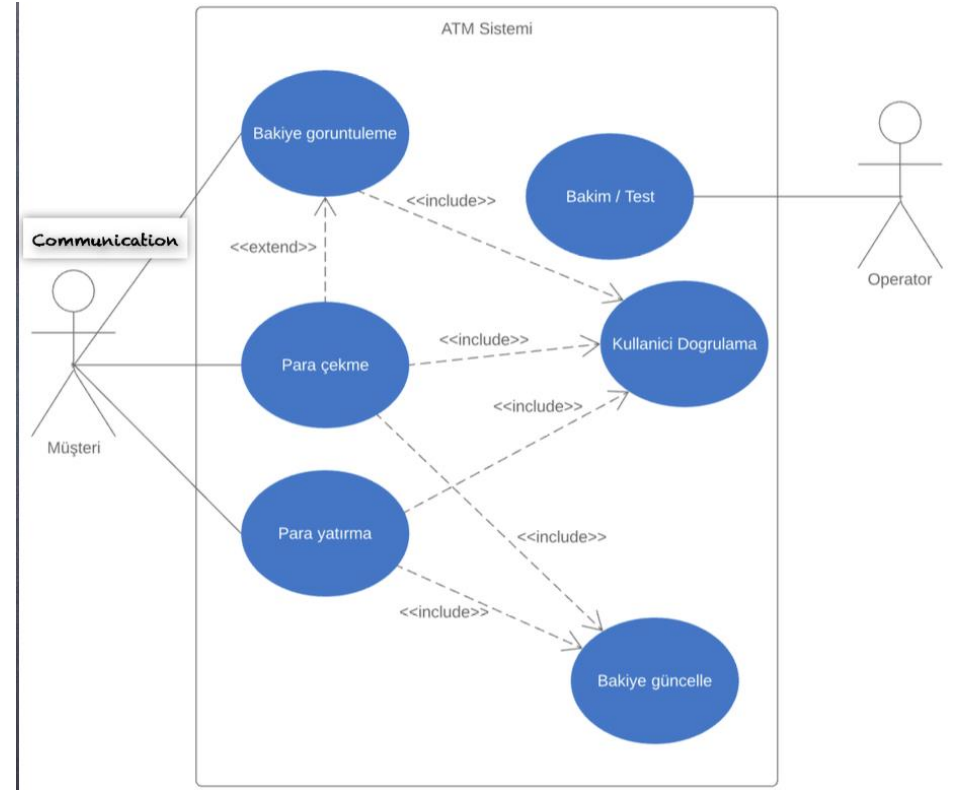
- Eşsiz bir ad, özet, oluşturma ve değiştirme tarihleri, sürüm numarası, sorumlu kişiler
- İlgili aktörler
- Giriş koşulu
- Çıkış koşulu
- Olay akışı
- Özel gereksinimler
 - yazılan şifreler görünmemeli vb.

Kullanım Durumları

- Para çekme
- Para yatırma
- Bakiye görüntüleme
- Parola değiştirme
- Para yükleme (Operatör)
- Sistem bakımı (Operatör)

Bağıntılar

- Communication: Aktör-system etkileşimi
- Include: Birden fazla kullanım durumu için gerekli işlevler (code reuse)
- Extends : istisna olarak ya da ara sıra gerçekleşen kullanım durumlarını ifade eder. Soyutlama adına (karmaşıklığı azaltmak) ayrı olarak verilebilir.
- Kalıtım: ortak işlemler için temel kullanım durumu tanımlanır ve kalıtım kullanılır



Generalization (Kalıtım)

- Aktörler arasında

Kullanıcı



| | |

Öğrenci Akademisyen Yönetici

- Use-case arasında

Ödeme Yap



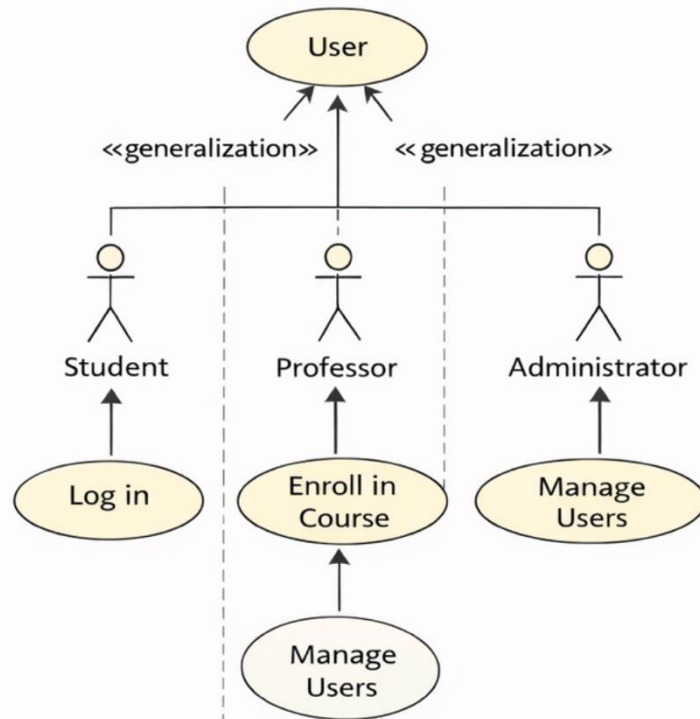
| | |

KrediKartı Havale QR
ile Ödeme ile Ödeme ile Ödeme

Inheritance in Use Case Diagrams

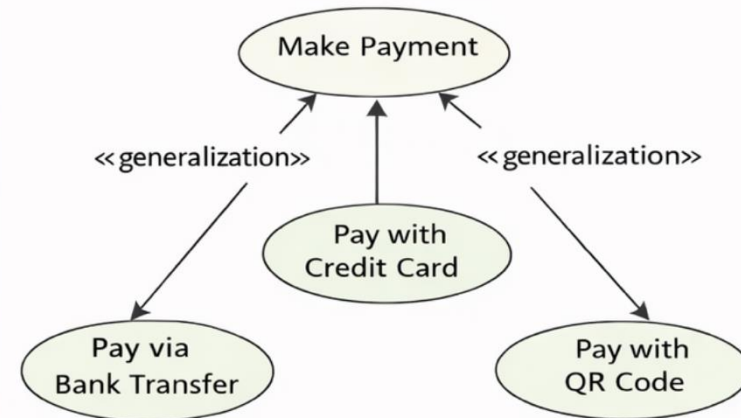
ACTOR INHERITANCE

Example 1: University System

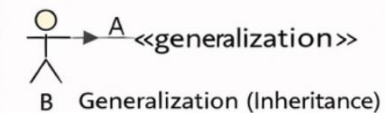


USE CASE INHERITANCE

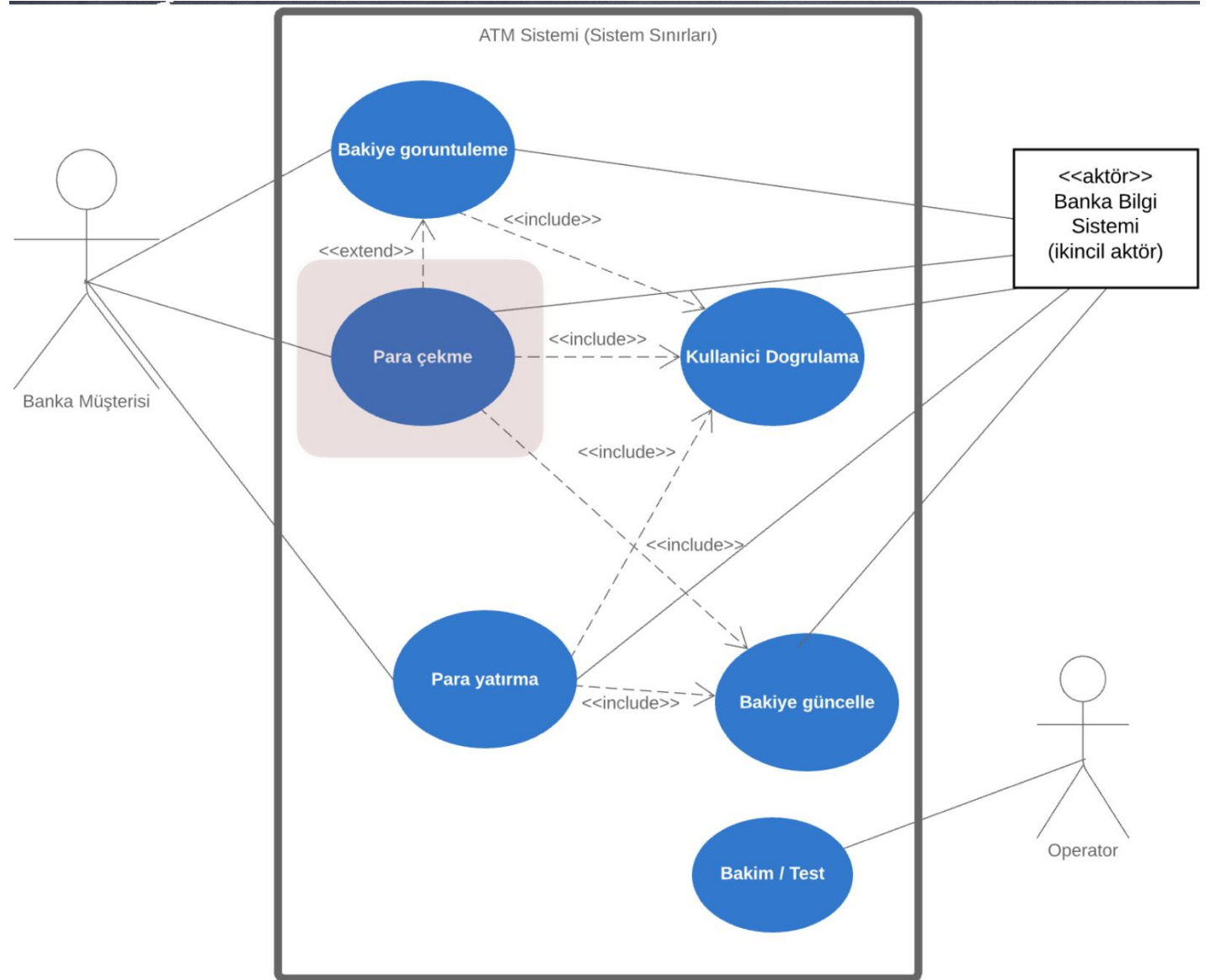
Example 2: Banking System



Legend:



Para Çekme Kullanım Durumu (Use Case)



Para Çekme Use Case Ayrıntıları

Kullanım durumunun tanımı

- Para çekme
- ATM den para çekme işlemini tanımlar

İlgili aktörler: Banka müşterisi, Banka BS (ikincil)

Giriş koşulu: Banka kartına sahip olan müşteri

Çıkış koşulu: Müşteri transactionı tamamlanır.

Olay akışı: Açıklanacak

Özel gereksinimler: UI gereksinimleri, işlem gecikmesi en fazla 1 dk olmalı, 24 saat çalışmalı

Olay Akışı-Başarılı

1. ATM sistemi Ekrana, müşteriden “kartını kart bölmesine takmasını” isteyen mesaj yazdırır.
2. Kart sahibi kartını kart bölmesine takar.
3. ATM sistemi kart doğrulamasını yapar.
4. Ekrana şifre girilmesini isteyen mesaj gönderilir.
5. Tuş takımını kullanarak girilen şifreyi alır.
6. Kullanıcı doğrulama ve yetki kontrolü için banka bilgi sistemine istek gönderilir.
7. Banka sistemi erişim isteğini kabul eder ve gerekli bilgileri gönderir.
8. ATM sistemi ekrana, çekilecek tutarları listeler.
9. Kart sahibi çekilecek tutarı girer.
10. ATM sistemi kartı çıkartır.
11. Kart sahibi kartı alır.
12. ATM sistemi parayı ve makbuzu verir.
13. Kart sahibi parayı ve makbuzu alır.

Olay Akışı-Alternatif akışlar

- **A1. Kart doğrulanamadı.**

4. Kart bölmesi kartı çıkartır.

5. İşlem sonlandırılır.

- **A2. Yanlış şifre**

- 8. 3 den az kez yanlış ise yeniden gir.

- 8. 3 kez yanlış girilmiş ise kart yutulur.

- 9. İşlem sonlandırılır.

- **A3. Yeterli para yok (9)**

- 10. ATM de yeterli bakiye yoksa yeni tutar girmesi istenir.

- 11. Hesapta yeterli bakiye yoksa yeni tutar girmesi istenir.

- **A4. Banka kartı alınmadı (10)**

- 11. 30 sn. boyunca alınmadı ise kart yutulur.

- 12. Gerekli bilgilendirme yapılır.

- 13. İşlem sonlandırılır.

- **A5. Para alınmadı (10)**

- 13. 30 sn. boyunca alınmadı ise kart yutulur.

- 14. Gönderilen ve yutulan para karşılaştırılır.

- 15. Hesap güncellemesi yapılır

- 12. Bilgilendirme yapılır.

- 13. İşlem sonlandırılır.

Sequence diyagramları

- Sequence (Sıralama) Diyagramı, UML'de sistem içindeki nesnelerin bir senaryo boyunca birbirleriyle hangi sırayla mesajlaştığını gösteren dinamik davranış diyagramıdır.

Ne anlatır?

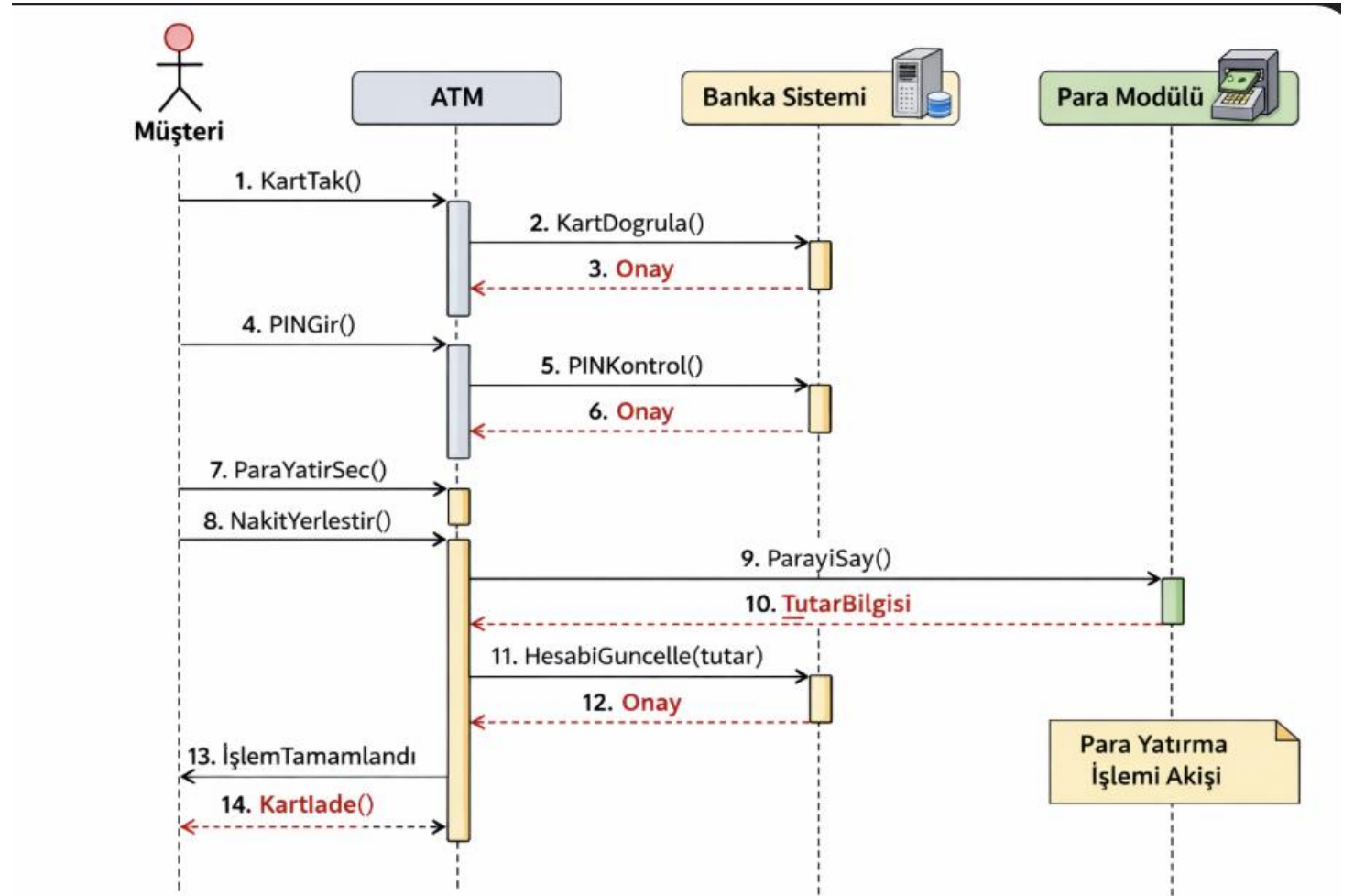
- Kim kiminle konuşuyor? Hangi nesne hangi nesneye mesaj gönderiyor?
- Hangi sırayla oluyor? İşlemler hangi zaman sırasıyla gerçekleşiyor?
- Hangi metod çağrılıyor? Her ok aslında bir metod çağrısını temsil eder.
- Sistem nasıl çalışıyor? Use case'te 'ne yapıldığı' yazarken, sequence diyagramında 'nasıl yapıldığı' gösterilir.

ATM örneği

- Use-case diyagramı şunu der:
- Kullanıcı para çeker.
- Sequence diyagramı şunu gösterir:
- Müşteri kartı takar
- ATM kartı bankaya doğrulatır
- Müşteri PIN girer
- Banka sistemi PIN'i kontrol eder
- ATM para çekme talebi gönderir
- Banka onay verir
- CashDispenser parayı verir

Sequence diyagramı temel elemanlar

- Aktör: Sistemi başlatan varlık
- Nesne: Sistem içindeki sınıfların örnekleri
- Lifeline: Nesnenin zaman içindeki varlığını gösteren dikey çizgi.
- Mesaj: Bir nesneden diğerine gönderilen ok.
- Activation Bar: Bir nesnenin aktif olarak işlem yaptığı süre.



Ne zaman kullanılır?

- Use-case'ı detaylandırmak için
- Sistem akışını netleştirmek için
- Yazılım geliştirilirken
- Hata analizinde

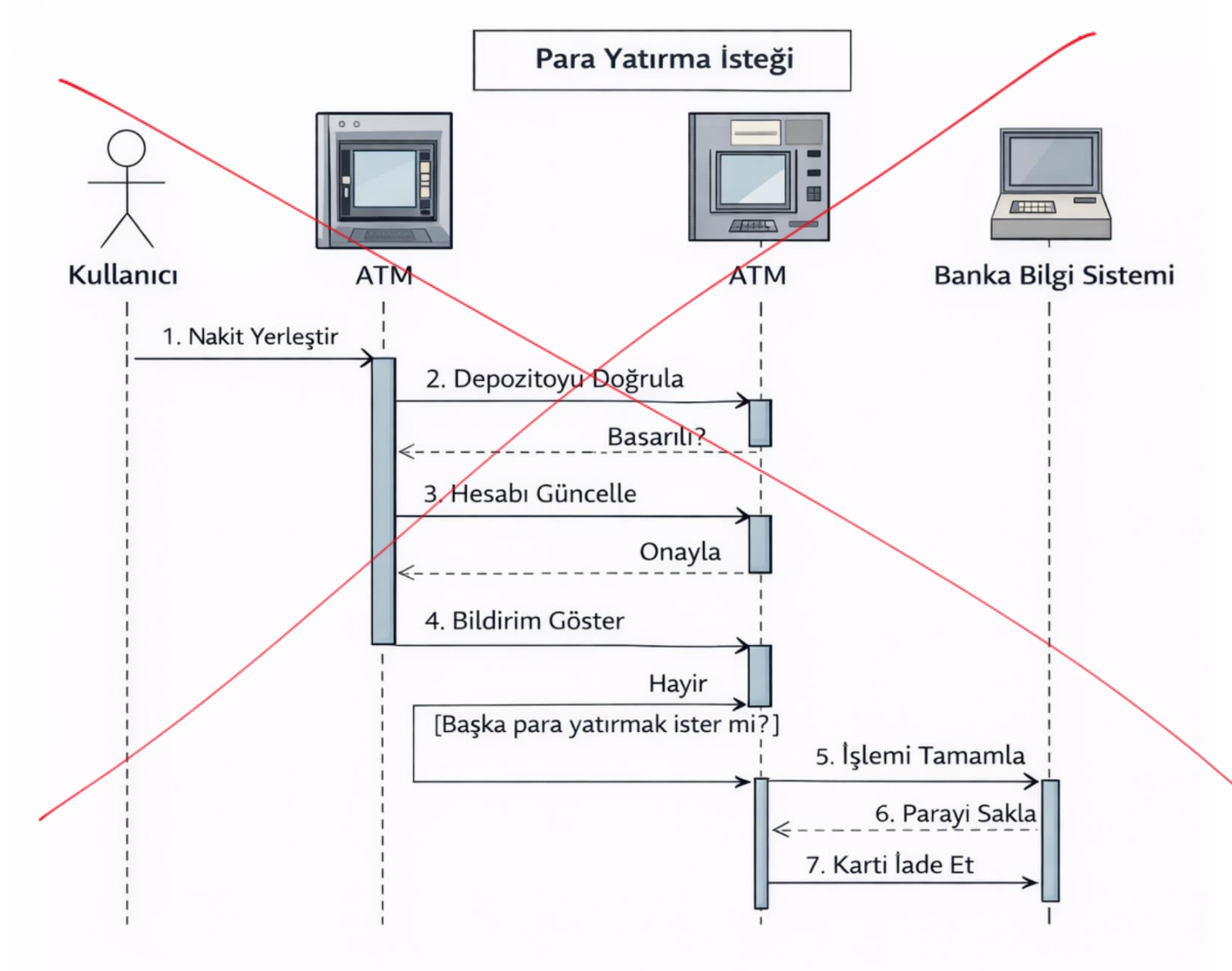
Ne zaman kullanılmaz?

- Sistem yapısını göstermek için kullanılmaz : Class tercih edilir.
- Sadece kullanıcı hedefini göstermek için: Use-case diyagramı kullanılır.

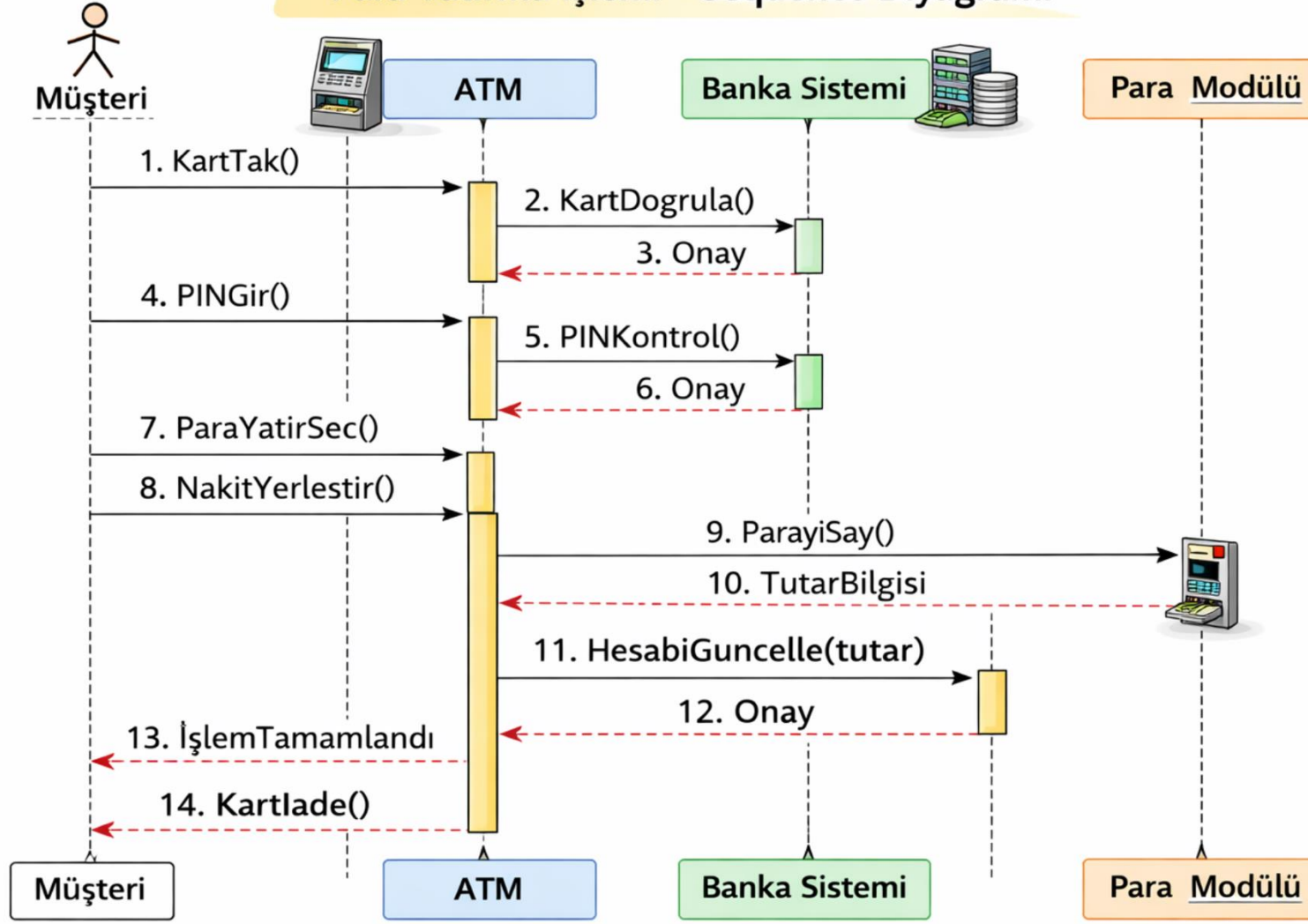
Para yatırma kullanım durumu sequence diyagramı

1. Müşteri → ATM : KartTak()
2. ATM → BankaSistemi : KartDogrula()
3. BankaSistemi → ATM : Onay
4. Müşteri → ATM : PINGir()
5. ATM → BankaSistemi : PINKontrol()
6. BankaSistemi → ATM : Onay
7. Müşteri → ATM : ParaYatirSec()
8. Müşteri → ATM : NakitYerlestir()
9. ATM → ParaModulu : ParayiSay()
10. ParaModulu → ATM : TutarBilgisi
11. ATM → BankaSistemi : HesabiGuncelle(tutar),
12. BankaSistemi → ATM : Onay
13. ATM → Müşteri : İşlemTamamlandı
14. ATM → Müşteri : Kartlade()

Para yatırma kullanım durumu sequence diyagramı



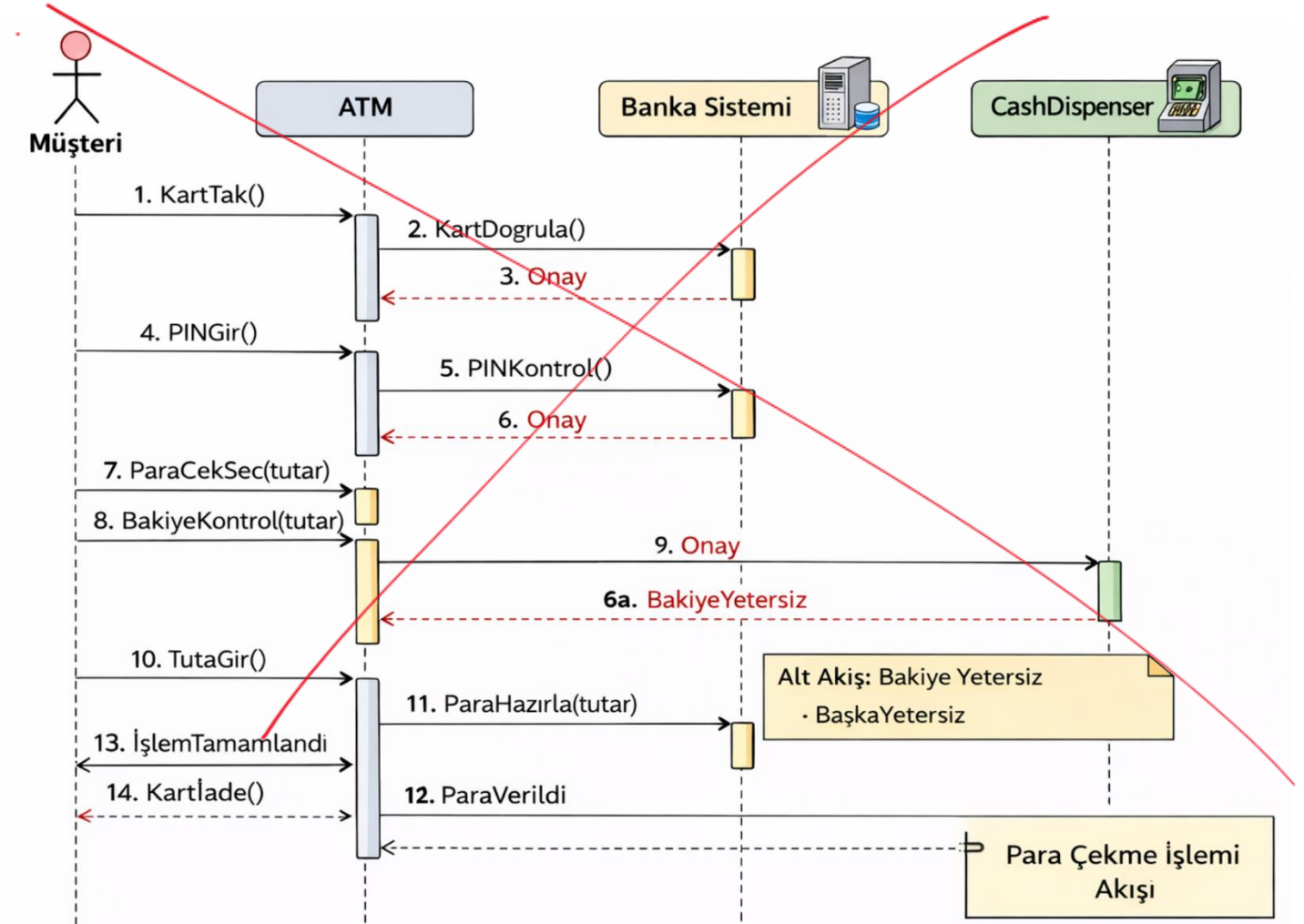
Para Yatırma İşlemi - Sequence Diyagramı



Para Çekme kullanım durumu sequence diyagramı

1. Müşteri → ATM : KartTak()
2. ATM → BankaSistemi : KartDogrula()
3. BankaSistemi → ATM : Onay
4. Müşteri → ATM : PINGir()
5. ATM → BankaSistemi : PINKontrol()
6. BankaSistemi → ATM : Onay
7. Müşteri → ATM : ParaCekSec(tutar)
8. ATM → BankaSistemi : BakiyeSorgula(tutar)
9. BankaSistemi → ATM : BakiyeDurumu
[bakiye yeterli]
10. ATM → CashDispenser : ParaHazirla(tutar)
11. CashDispenser → ATM : ParaVerildi
12. ATM → BankaSistemi : HesapGuncelle(tutar)
13. BankaSistemi → ATM : Onay
[bakiye yetersiz]
10. ATM → Müşteri : HataMesajiGoster()
14. ATM → Müşteri : İşlemTamamlandı
15. ATM → Müşteri : Kartlade()

Para çekme kullanım durumu sequence diyagramı



Alt akış kutusu

Combined Fragment

- Altında bir operatör yazar:
- alt → alternative (koşullu akış)
- opt → optional
- loop → döngü
- par → paralel

**UML'de not koyulacaksa
eğer;**

+-----+

| note |

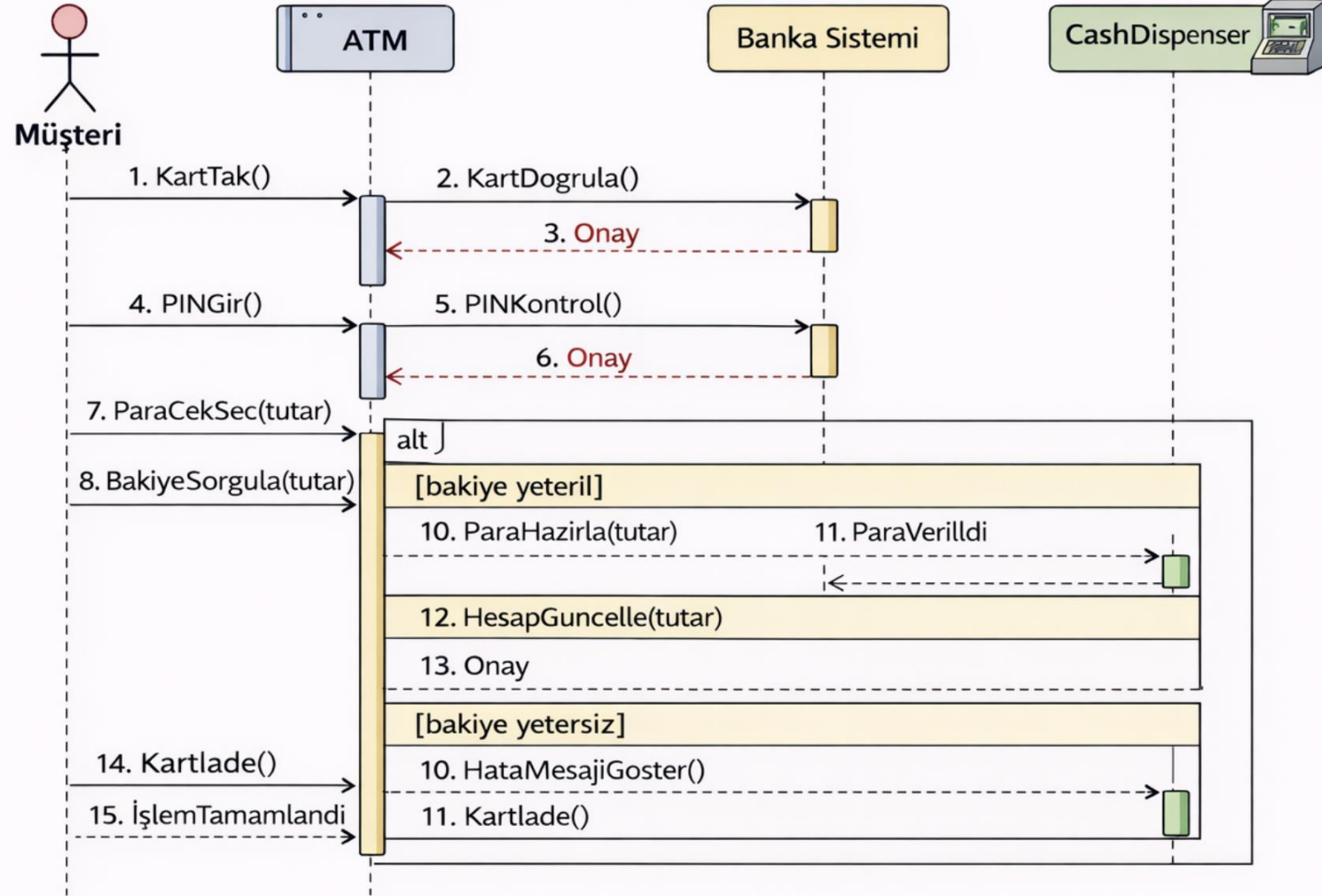
| Açıklama |

+-----+

|

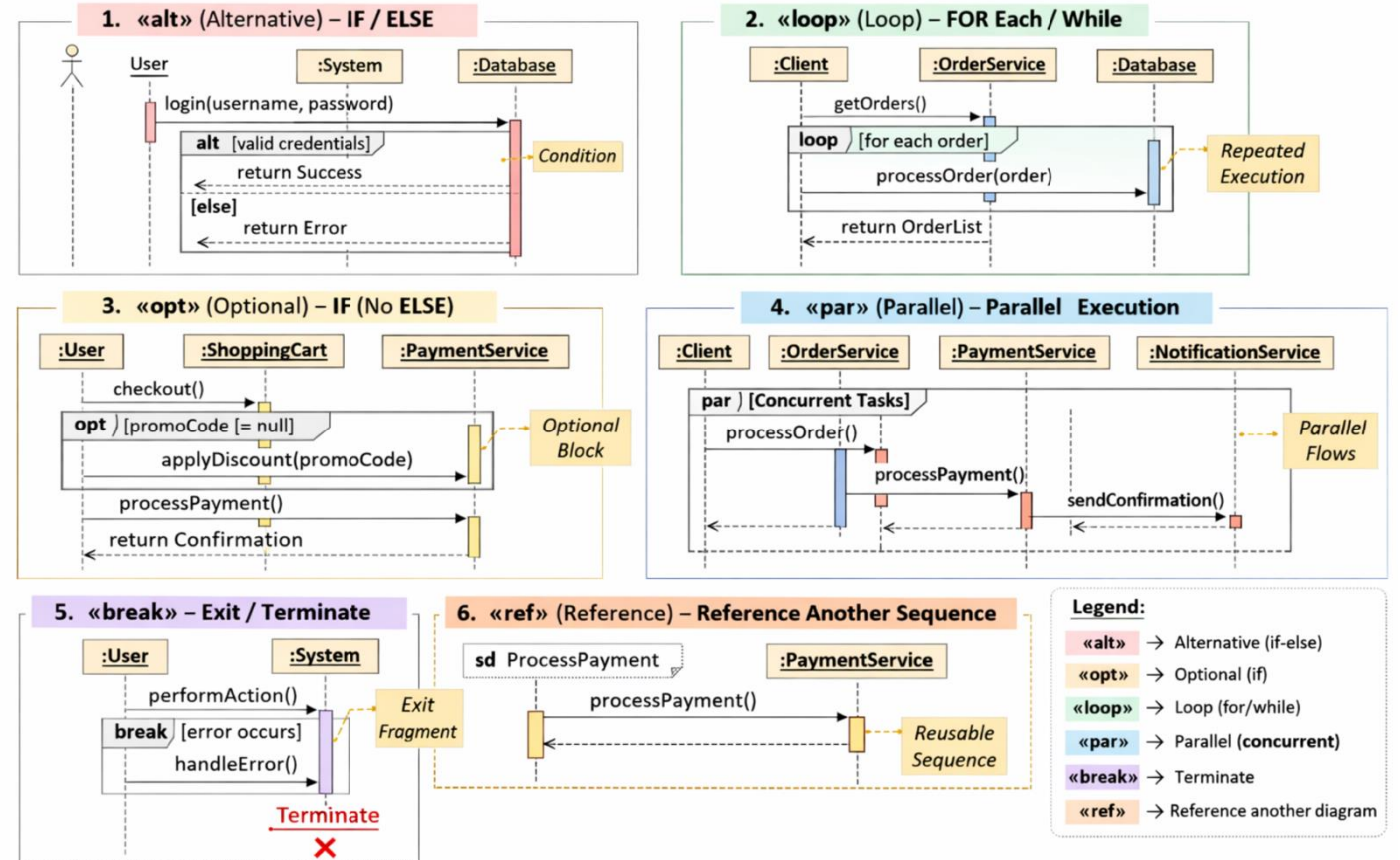
|---- ilgili elemana bağlanır

Para Çekme İsteği



Combined Fragment

UML Sequence Diagram – Combined Fragments Examples



Use-case olay akışı -Sequence diyagramı

1. ATM sistemi Ekrana, müşteriden “kartını kart bölmesine takmasını” isteyen mesaj yazdırır.
2. Kart sahibi kartını kart bölmesine takar.
3. ATM sistemi kart doğrulamasını yapar.
4. Ekrana şifre girilmesini isteyen mesaj gönderilir.
5. Tuş takımını kullanarak girilen şifreyi alır.
6. Kullanıcı doğrulama ve yetki kontrolü için banka bilgi sistemine istek gönderilir.
7. Banka sistemi erişim isteğini kabul eder ve gerekli bilgileri gönderir.
8. ATM sistemi ekrana, çekilecek tutarları listeler.
9. Kart sahibi çekilecek tutarı girer.
10. ATM sistemi kartı çıkartır.
11. Kart sahibi kartı alır.
12. ATM sistemi parayı ve makbuzu verir.
13. Kart sahibi parayı ve makbuzu alır.

Örnek: E-Ticaret Uygulaması

Belirtilen gereksinimlere (iş kurallarına) göre kullanım durum diyagramı oluşturunuz.

Senaryo

Elektornik ticaret yazılımının ihtiyacı olan verilerin yönetimi için bir veritabanı geliştirilmesi isteniyor. İş kuralları verilen bu veritabanının VB diyagramını ve ilişkisel şemasını oluşturunuz.

İş Kuralları

- Bu veritabanında her müşteriye bir numara verilerek müşterinin TC Kimlik numarası, adı, soyadı, yaşadığı ili saklanması düşünülüyor.
- İllerin plaka numarası ve adı saklanır.
- Her siparişe bakan bir satış temsilcisi mevcuttur. Satış temsilcilerinin TCKimlikNo, ad ve soyad bilgileri mevcuttur.
- Ürünlerin (kişisel bilgisayar, telefon vb.) kodu, adı, fiyatı ve stok miktarlarının saklanması gerekmektedir.
- Ürünlerin kategorileri (bilgisayar, ev elektroniği, kozmetik vb.) mevcuttur.
- Müşterilerin ürün siparişleri saklanarak her bir siparişe bir fatura kesilmesi sağlanmalıdır.
- Her siparişin eşsiz bir sipariş numarası ve sipariş tarihi mevcuttur.
- Sipariş edilen bir ürünün sipariş adedi ve birim fiyatı (kişiyeye özel indirim v.s. nedeniyle ürün tablosundaki fiyattan farklılık gösterebilir) da kaydedilmelidir.
- Faturaların fatura numarası, tarih ve fatura adresi bilgileri saklanmalıdır. (Toplam fiyat hesaplanabilir ya da saklanabilir).
- Siparişler bir kargo firması tarafından iletilir. Kargo firmasının kodu, adı, adresi bilgileri yer alır. Her kargo firmasında siparişlerden sorumlu bir yetkili yer alır.
- Bir ürünün yalnızca bir kategorisi mevcuttur. Bir kategori çok sayıda ürünün kategorisi olabilir.
- Bir siparişte en az bir ürün bulunur. Ancak çok sayıda ürün de bulunabilir. Bir ürün çok sayıda siparişte yer alabilir.
- Bir müşteri çok sayıda sipariş verebilir. Bir sipariş yalnızca bir müşteri tarafından verilebilir.
- Bir siparişin yalnızca bir faturası olabilir. Bir fatura yalnızca bir siparişin faturası olabilir.
- Bir sipariş ile yalnızca bir satış temsilcisi ilgilenir. Bir satış temsilcisi çok sayıda sipariş ile ilgilenebilir.
- Bir müşteri yalnızca bir ilde yaşayabilir. Bir ilde çok sayıda müşteri yaşayabilir.
- Bir sipariş yalnızca bir kargo firması tarafından iletilir. Bir kargo firması çok sayıda sipariş iletebilir.

<https://github.com/celalceken/DatabaseManagementSystems/blob/master/VYS04.md>